SE5104
Mini Project - Project Proposal



FoodLence - AI-Based Food Label Scanner
Mobile App Development

Group Members:

H.G.B.A Mekala – 22CSE0361
H.G.K Rashmika – 21CSE0190
S.K.P.H Deemantha – 22CSE0348
P.A.A Dilshan – 22CSE0360

Internal Supervisor : Ms. B.R. Madhurangi
Mentor : Mr. G.A.C.G. Dhanawardhana

ACADEMIC YEAR: 2022 / 2023
GROUP 14
DEPARTMENT OF SOFTWARE ENGINEERING
FACULTY OF COMPUTING
SABARAGAMUWA UNIVERSITY OF SRI LANKA

Approval of Mini Project

01. Title of the Mini Project : AI-Based Food Label Scanner – Personalized Allergen & Ingredient Understanding

02. Details of Group Members :

Index No	Name with Initials	Email	Mobile No	Signature
22CSE0361	H.G.B.A Mekala	22cse0361@ms.sab.ac.lk	0742368019	
21CSE0190	H.G.K Rashmika	hgkrashmika@std.foc.sab.ac.lk	0710123837	
22CSE0348	S.K.P.H.Deemantha	22cse0348@ms.sab.ac.lk	0776171219	
22CSE0360	P.A.A Ishara Dilshan	22cse0361@ms.sab.ac.lk	0775944600	

03. Name of the Internal Supervisor : Ms. B.R. Madhurangi

04. Internal Supervisor's Designation : Lecturer

05. Name of the Mentor : Mr. G.A.C.G. Dhanawardhana

06. Mentor's Designation : Software Engineer

07. Mentor's Organization : LSEG Technology

08. Mentor's Signature :

For office use only:

Approved / Not approved : ……

Signature of the Internal Supervisor : ……

Date : 9th September 2026

Suggestions if any : ……

Table of Contents

Approval of Mini Project 1
LIST OF TABLES 5
LIST OF FIGURES 5
LIST OF ABBREVIATIONS 6
1. TEAM DETAILS AND PROPOSED MODULE OWNERSHIP 7
1.1. H.G.B.A. Mekala — Mobile & Ui Lead (22cse0361) 7
1.2. H.G.K. Rashmika — Ml Dataset & Training Lead (21cse0190) 8
1.3. S.K.P.H. Deemantha — Knowledge Base Lead (22cse0348) 8
1.4. P.A.A. Ishara Dilshan — Backend & Deployment Lead (22cse0360) 9
1.5. Integration Points 9
2. INTRODUCTION AND PROBLEM DEFINITION 10
2.1. Background And Context 10
2.2. Target Users And Stakeholders 10
2.3. Specific Problem Being Addressed 11
2.4. Limitations Of Existing Approaches 11
2.5. Motivation And Significance 12
3. EXISTING SOLUTIONS AND GAP ANALYSIS 13
3.1. Existing Approaches And Systems 13
3.2. Strengths And Limitations Of Existing Solutions 13
3.3. The Capability Gap 14
3.4. Unaddressed Needs And How This Project Addresses Them 14
4. PROJECT AIM, OBJECTIVES AND SCOPE 15
4.1. Project Aim 15
4.2. 4.2 Project Objectives 15
4.3. Scope 16
4.3.1. In Scope 16
4.3.2. Out of Scope 17
5. REQUIREMENTS ANALYSIS 18
5.1. Functional Requirements 18
5.2. Non-Functional Requirements 19
6. PROPOSED SOLUTION 21
6.1. Overall Solution Concept (Client-Server Shift) 21
6.2. Major Features 21
6.3. System Workflow (Edge OCR to Cloud NLP) 22
6.4. User Interaction 23
6.5. Major Inputs and Outputs 23
6.6. External Systems and Services 24
6.7. Integration Between Major Components 24
7. SYSTEM ARCHITECTURE AND TECHNICAL DESIGN 25
7.1. High-Level Client-Server System Architecture 25
7.2. Major Components and Modules 25
7.3. Interfaces (RESTful APIs) 26

7.4. Data Flow (Mobile to FastAPI to PostgreSQL) 26
7.5. Cloud Database Schema (PostgreSQL) 26
7.6. API and Service Interfaces 29
7.7. Client, Server, and Device Relationships 29
7.8. Deployment Environment 30
7.9. Relevant Diagrams 30
8. TECHNOLOGY SELECTION AND JUSTIFICATION 34
8.1. Programming Languages 34
8.2. Frameworks and Libraries 34
8.3. Database and Storage 35
8.4. Platforms and Development Tools 36
8.5. Cloud and Deployment Technologies 36
9. TECHNICAL DEPTH AND CORE ENGINEERING WORK 37
9.1. ML Dataset Creation and Annotation (Handling OCR Noise) 37
9.2. NLP Classification Model Training and Optimization 37
9.3. Semantic Translation and Allergen Engine Implementation 38
9.4. FastAPI Backend and Inference Integration 38
10. DATA, SECURITY, PRIVACY AND ETHICAL CONSIDERATIONS 40
10.1. Types and Sources of Training Data 40
10.2. Personal and Sensitive Information 40
10.3. Authentication, Authorization, and Data Protection 40
10.4. Ethical and Legal Considerations (Medical Disclaimer) 41
11. TESTING AND EVALUATION PLAN 42
11.1. Unit and Integration Testing 42
11.2. Machine Learning Evaluation (F1-Score, Precision, Recall) 42
11.3. System and Performance Testing (Backend Latency) 43
11.4. User Acceptance Testing 43
12. DEVELOPMENT PROCESS AND GITHUB PLAN 45
12.1. Development Workflow and Branching Strategy 45
12.2. Integration Approach and Code Review 45
12.3. GitHub Repository Details 45
13. PROJECT MANAGEMENT PLAN 47
13.1. Project Management Tool (Jira) 47
13.2. Development Phases (Incorporating Dataset Collection & Model Training) 47
Phase 1: Project Setup and UI Prototyping (Week 1) 47
Phase 2: Edge OCR Pipeline and Noisy Data Collection (Weeks 2-3) 47
Phase 3: ML Model Training and Database Schema Construction (Weeks 4-6) 47
Phase 4: Client-Server API Integration (Weeks 7-9) 48
Phase 5: Multi-Profile Logic and Advanced UI (Weeks 10-11) 48
Phase 6: System Evaluation and User Acceptance Testing (Weeks 12-13) 48
Phase 7: Deployment and Academic Documentation (Weeks 14-15) 48
13.3. Major Activities, Milestones, and Module Responsibilities 48
14. FEASIBILITY AND RISK ANALYSIS 50

14.1. Technical and Performance Risks 50
14.2. Machine Learning and Data Risks 50
14.3. Project Management and Schedule Risks 51
14.4. Legal and Ethical Risks 51
15. RESOURCE ALLOCATION AND BUDGET ESTIMATION 52
15.1. Human Resources 52
15.2. Hardware Requirements 52
15.3. Software and Cloud Infrastructure 52
15.4. Total Estimated Budget 53
16. PROJECT DELIVERABLES AND EXPECTED OUTCOMES 54
16.1. Primary System Deliverables 54
16.2. Expected Outcomes and Impact 54
16.3. Future Enhancements 55
CONCLUSION 56
REFERENCES 57

LIST OF TABLES

Table 1: Group Members And Module Ownership....7
Table 2: Users Database Schema....27
Table 3: Profiles Database Schema....27
Table 4: Ingredients Database Schema....27
Table 5: Ingredients Database Schema....27
Table 6: Ingradient_Allergen_Mapping Database Schema....28
Table 7: Additives Database Schema....28
Table 8: Inference Log Database Schema....29

LIST OF FIGURES

Figure 1: High Level System Architecture....30
Figure 2: DFD Level 0....31
Figure 3: DFD Level 01....31
Figure 4: ER Diagram....32
Figure 5: Sequence Diagram....32
Figure 6: Use Case Diagram....33
Figure 7: Mobile UI Wireframe....33

LIST OF ABBREVIATIONS

- AI – Artificial Intelligence
- API – Application Programming Interface
- APK – Android Application Package
- CRUD – Create, Read, Update, Delete
- DFD – Data Flow Diagram
- ER – Entity-Relationship
- JWT – JSON Web Token
- ML – Machine Learning
- MVP – Minimum Viable Product
- NLP – Natural Language Processing
- OCR – Optical Character Recognition
- OTC – Over-The-Counter
- PDPA – Personal Data Protection Act
- RBAC – Role-Based Access Control
- SDK – Software Development Kit
- SSE – Server-Sent Events
- SQL – Structured Query Language
- SSL – Secure Sockets Layer
- TLS – Transport Layer Security
- UI – User Interface
- UAT – User Acceptance Testing
- UX – User Experience

1. TEAM DETAILS AND PROPOSED MODULE OWNERSHIP

The AI-Based Food Label Scanner follows a client-server machine learning architecture: the Flutter mobile app captures label images, runs on-device OCR (Google ML Kit), and sends the extracted text to a Python (FastAPI) cloud backend. A custom-trained NLP classification model predicts the correct ingredient entity from the noisy OCR text, and a PostgreSQL knowledge base translates the predicted entity into allergen warnings. Module boundaries follow this pipeline, with each member owning one layer.

Group Member	Assigned Modules	Contribution
H.G.B.A. Mekala (Mobile & UI Lead) 22CSE0361	Flutter app, camera scanner, OCR invocation, results dashboard, scan history	25%
H.G.K. Rashmika (ML Dataset & Training Lead) 21CSE0190	OCR-noise dataset creation, annotation, model training, evaluation, model serialisation	25%
S.K.P.H. Deemantha (Knowledge Base Lead) 22CSE0348	PostgreSQL schema, chemical/E-number to allergen mapping, translation queries	25 %
P.A.A. Ishara Dilshan (Backend & Deployment Lead) 22CSE0360	FastAPI backend, inference endpoint, JWT auth, cloud deployment	25%

Table 1: Group Members And Module Ownership

1.1. H.G.B.A. MEKALA — MOBILE & UI LEAD (22CSE0361)

- **Owns the Flutter client.**

Responsibilities:

- camera scanner interface (focus guide, flash toggle, capture/retake).
- Google ML Kit OCR integration producing raw text.
- results dashboard rendering the backend's colour-coded allergen warnings.
- scan history with local persistence.
- onboarding and medical disclaimer screens.

Deliverables:

- functional Flutter app (Android 8.0+)
- OCR-to-backend integration

- Figma designs
- UI integration tests against the API contract.

1.2. H.G.K. RASHMIKA — ML DATASET & TRAINING LEAD (21CSE0190)

➢ **Owns the machine learning asset.**

Responsibilities:

- constructing the training dataset from real OCR outputs plus synthetic noisy variants (e.g., wh ey → whey)
- annotating noisy text to canonical ingredient entities
- spaCy + TF-IDF preprocessing with character n-gram features
- training and benchmarking classifiers (Linear SVM, Logistic Regression, Complement Naive Bayes) with cross-validation and hyperparameter tuning
- evaluation via precision, recall, and F1-score with error analysis.

Deliverables:

- annotated dataset (3,000+ samples, 150-200 classes)
- serialised model artefact
- evaluation report targeting macro - $F_1 \geq 0.85$.

1.3. S.K.P.H. DEEMANTHA — KNOWLEDGE BASE LEAD (22CSE0348)

➢ **Owns the semantic translation layer.**

Responsibilities:

- PostgreSQL schema (entities, synonyms, E-numbers, allergen groups).
- Curating mappings from FDA, EFSA, and CODEX covering the nine major allergens plus the sulfite group (E220-E228 → sulfite warning; casein/whey/E322 → milk).
- Query layer resolving a predicted entity into allergen group, additive category, and plain-language explanation.

Deliverables:

- Populated knowledge base (300+ entities, 100+ additives)
- Translation query layer
- Curation documentation.

1.4. P.A.A. ISHARA DILSHAN — BACKEND & DEPLOYMENT LEAD (22CSE0360)

- **Owns the service layer.**

Responsibilities:

- Fastapi backend with Pydantic schemas and auto-generated openapi docs.
- Inference endpoint loading the serialised model and returning predictions with confidence scores.
- Integration of knowledge base translation into the response pipeline.
- JWT authentication with bcrypt; bridging real OCR outputs to the dataset team.
- Docker containerisation and cloud deployment (Render/Railway) with CI/CD.

Deliverables:

- Deployed backend with documented endpoints.
- Inference service.
- Auth system.
- Deployment documentation.

1.5. INTEGRATION POINTS

The four workstreams converge at two frozen contracts, defined in Phase 1-2:

(i) REST API payload contract	client ↔ backend
(ii) Model artefact contract	training pipeline ⟶ inference

These contracts allow all four members to develop in parallel, with integration tested incrementally at each sprint boundary.

2. INTRODUCTION AND PROBLEM DEFINITION

2.1. BACKGROUND AND CONTEXT

Nearly every packaged food product carries a printed ingredient label, yet these labels are increasingly difficult for ordinary consumers to understand. Manufacturers use scientific names, chemical codes, and regional terminology: a person allergic to milk may not recognise "caseinate", "whey concentrate", or "E322" as milk-derived; a person allergic to eggs may not know that "albumin" or "lysozyme" are egg proteins. This gap between what is printed on the label and what the consumer understands is a direct health risk for people with food allergies, where a single overlooked ingredient can trigger a severe reaction.

Smartphones now offer high-quality cameras, and machine learning has matured to the point where text can be understood in context, not merely read character by character. This project combines on-device OCR, a cloud-hosted NLP classification model, and a curated chemical knowledge base to build an application that scans a food label, interprets the ingredients despite OCR noise, and delivers a personalised allergen warning within seconds.

2.2. TARGET USERS AND STAKEHOLDERS

- **Allergy sufferers**
  Individuals who must verify every product before consumption; a labelling mistake can have serious consequences.

- **Parents and guardians**
  Must check every snack and meal for one or more children, each with different allergies; benefit from family profiles.

- **Elderly users**
  Struggle with small print and unfamiliar terminology; benefit from camera-based scanning instead of manual reading.

- **Caregivers and healthcare**
  Schools, daycare, and care homes must check workers products against multiple dietary restrictions quickly and reliably.

- **Health-conscious consumers**

  Wish to avoid specific additives, colourants, or preservatives even without a diagnosed allergy.

- **Stakeholders**

  The development team, internal supervisor, the department, and potential future partners in healthcare or retail.

2.3. SPECIFIC PROBLEM BEING ADDRESSED

The project addresses four interlocking sub-problems:

1. Text extraction

   Reading ingredient text from curved, reflective, low-contrast packaging using a phone camera (handled on-device by ML Kit).

2. Noisy text understanding

   Interpreting misspelled, fragmented OCR output and identifying the correct ingredient entity (handled by the ML classification model on the backend).

3. Semantic translation

   Resolving the predicted entity to allergen groups via chemical/E-number mappings (handled by the PostgreSQL knowledge base).

4. Personalised alerting

   Matching detected allergens against individual or family profiles and displaying an unambiguous, colour-coded result fast enough for in-store use.

2.4. LIMITATIONS OF EXISTING APPROACHES

- Barcode-scanning apps depend on large product databases. Local Sri Lankan brands and small manufacturers are absent, and manufacturer recipe changes make entries stale. The app is useless when the product is unlisted.

- General-purpose OCR tools (Google Lens, Adobe Scan) extract text but understand nothing: they return "E220" without warning the user that it is a sulfiting agent.

- Manual web searching of each ingredient is accurate but takes many minutes per product - impractical during routine shopping.

- Simple allergen checklist apps perform exact keyword matching only, missing hidden allergens (whey, casein, E322 for a milk allergy) and giving users a false sense of security.

- None of these approaches personalise results across multiple family profiles or explain ingredients in plain language.

2.5. MOTIVATION AND SIGNIFICANCE

Food allergy prevalence is rising globally, and Sri Lanka lacks dedicated tools: consumers rely on manual label reading, a process that is slow, error-prone, and stressful. An application that reads the physical label rather than a product database works for any packaged food regardless of brand, country of origin, or database coverage - a decisive advantage in a market dominated by local and regional products.

For the development team, the project carries substantial technical depth: it requires an end-to-end machine learning pipeline (dataset creation, annotation, training, evaluation), a production backend serving a trained model, a relational knowledge base for semantic translation, and a polished mobile client - integrated into one coherent system.

For users, it converts label reading from a manual, expert-level task into a point-and-scan decision, reducing anxiety and improving food safety for allergy sufferers, parents, and caregivers.

3. EXISTING SOLUTIONS AND GAP ANALYSIS

3.1. EXISTING APPROACHES AND SYSTEMS

Four categories of tools currently attempt to help consumers understand food labels:

- Barcode-scanning nutrition apps (Yuka, Fooducate, Open Food Facts) look up the product's barcode in a central database and display pre-recorded ingredients, nutrition data, and sometimes allergen flags.

- General-purpose OCR tools (Google Lens, Adobe Scan, Microsoft Lens) extract text from any image, with limited translation or web-search features, but no food-domain understanding.

- Manual search (Google, medical and food-science websites) gives accurate per-ingredient information, but requires the user to type and research every unfamiliar term.

- Simple allergen checklist apps let users select allergens and match typed ingredient text against a small keyword list - basic and offline, but brittle.

3.2. STRENGTHS AND LIMITATIONS OF EXISTING SOLUTIONS

✓ Barcode apps are fast and polished - but only when the product exists in the database. Coverage of Sri Lankan and regional brands is poor, and stale entries create false safety assurances when manufacturers reformulate.

✓ OCR tools are highly accurate at reading text but provide no semantic interpretation: reading "E220" or "casein" triggers a web search at best, never a personalised allergen warning.

✓ Manual search is authoritative but takes many minutes per product, making it impractical for routine shopping.

✓ Checklist apps are lightweight and offline, but exact keyword matching misses hidden allergens entirely (whey, casein, E322 for a milk allergy), producing dangerous false negatives.

✓ The common weakness across all four

- None combine direct label reading.
- Intelligence that survives ocr noise and synonymy.
- And personalisation.

3.3. THE CAPABILITY GAP

Ingredient names on real labels form a many-to-many mapping: one allergen group appears as dozens of chemical names, synonyms, regional spellings, and OCR corruptions. Consumer tools today handle at most one side of this problem - they either read the text without understanding it (OCR tools), or understand allergens only for pre-catalogued products (barcode apps). No available tool interprets arbitrary printed label text and translates it into personalised allergen warnings.

3.4. UNADDRESSED NEEDS AND HOW THIS PROJECT ADDRESSES THEM

- **Universal coverage**
  Reads the printed label directly, so it works for local brands, imports, and unlisted products that barcode apps cannot serve.

- **Noisy-text understanding**
  A machine learning classification model trained on OCR noise (misspellings, splits, case errors) resolves the correct ingredient entity where literal matching fails (Section 8).

- **Semantic translation**
  A relational knowledge base maps predicted entities to allergen groups - for example E220-E228 and metabisulfite resolving to a sulfite warning (Section 9).

- **Multi-profile checking**
  One scan evaluated against several family profiles, showing who can safely eat the product.

- **Additive education**
  Plain-language explanations for preservatives, colourants, and E-numbers, not just safe/unsafe flags.

Fast, personalised, colour-coded results delivered within seconds - none of the existing categories delivers all of these together.

4. PROJECT AIM, OBJECTIVES AND SCOPE

4.1. Project Aim

The primary aim of this project is to engineer a robust, machine learning-driven Client-Server application that automates the extraction, classification, and semantic translation of packaged food ingredient labels to generate personalized allergen warnings. By integrating edge-based Optical Character Recognition (OCR) with a cloud-hosted Natural Language Processing (NLP) pipeline, the system bridges the gap between complex food chemistry terminology and consumer safety, mitigating the risks associated with manual label reading.

4.2. 4.2 Project Objectives

To successfully transition from a rudimentary rule-based prototype to a sophisticated machine learning architecture, the project must achieve the following specific, measurable objectives:

- **Objective 1: Edge-Based Image Capture and Text Extraction.**
  Develop a cross-platform mobile client (Flutter) utilizing Google ML Kit to handle real-time image capture, preprocessing, and on-device OCR text extraction under varying real-world lighting and physical label conditions.

- **Objective 2: ML Dataset Creation and Annotation.**
  Curate, clean, and annotate a comprehensive dataset consisting of noisy, real-world OCR-extracted ingredient text, mapping these flawed inputs to clean, ground-truth ingredient entities to facilitate supervised machine learning.

- **Objective 3: Cloud-Hosted NLP Classification Engine.**
  Train, validate, and deploy a custom Natural Language Processing classification model utilizing scikit-learn and spaCy within a Python environment. The model must accurately predict specific ingredient entities from unstructured, erroneous OCR strings, achieving a high F1-score.

- **Objective 4: Relational Knowledge Base Construction.**
  Design a highly normalized PostgreSQL database to store and semantically map complex chemical names, alternative terminologies, and E-numbers to the nine major internationally recognized allergen groups (milk, egg, peanut, tree nuts, soy, wheat, fish, shellfish, and sesame).

- **Objective 5: Semantic Translation and Personalized Evaluation.**
  Implement a high-performance FastAPI backend to orchestrate the pipeline, cross-referencing predicted ingredient entities against the PostgreSQL knowledge base, and evaluating them against user-specific allergen profiles to generate deterministic safety warnings.

- **Objective 6: Multi-Profile Dietary Management.**
  Engineer a secure user management module that allows individuals, parents, and caregivers to maintain up to five customized dietary profiles simultaneously, facilitating bulk checking from a single scan.

- **Objective 7: Interactive Result Visualization.**
  Design an intuitive mobile UI to present color-coded safety classifications (red for danger, yellow for caution, green for safe), plain-language additive explanations, and maintain a local historical log of scanned products.

- **Objective 8: System Evaluation and Metric Optimization.**
  Rigorously evaluate the system architecture by assessing the ML model's precision, recall, and F1-score, alongside system-wide backend latency and OCR accuracy under constrained conditions.

4.3. Scope

4.3.1. In Scope

- **Major Functions:**
  - Camera-based label capture.
  - automated OCR extraction.
  - ML-driven ingredient text classification.
  - semantic mapping to nine major allergens.
  - additive detection with plain-language context.
  - creation of personal/family allergen profiles.
  - manual text-search fallback.
  - and local scan history storage.

- **System Components:**
  - Flutter mobile frontend.
  - Google ML Kit edge processing module, .
  - Python/FastAPI cloud backend.
  - scikit-learn and spaCy NLP pipeline.
  - PostgreSQL relational database.
  - and JWT-based user authentication module.

- **Target Audience:**
  - Individuals with diagnosed food allergies.
  - parents and guardians.
  - elderly users.
  - healthcare caregivers.
  - and health-conscious consumers.

- **Platform Support:**
  Android Operating System (version 8.0 and above) requiring a minimum 8-megapixel rear camera.

- **Performance Baselines:**
  The system must achieve a minimum of 85% character accuracy on clear labels and process the entire scan-to-result pipeline efficiently to deliver actionable UI warnings within an acceptable timeframe for active shoppers.

4.3.2. Out of Scope

- **Offline AI Processing:**
  Due to the architectural shift to a Client-Server model, all machine learning inference and semantic matching require active internet connectivity to query the FastAPI cloud backend. Strict offline AI capability is excluded.

- **Multilingual Processing:**
  The initial MVP restricts OCR extraction and NLP classification strictly to English-language label structures. Support for Sinhala and Tamil is reserved for future iterations.

- **Barcode and QR Code Lookups:**
  The system relies exclusively on direct physical text analysis via OCR. Scanning barcodes to retrieve pre-existing product data from external cloud databases is not supported.

- **Nutritional Analysis:**
  Detailed macronutrient breakdowns, caloric counting, vitamin tracking, and generalized diet planning are excluded to maintain strict focus on allergens and additives.

- **Medical Diagnosis:**
  The application serves strictly as an informational support tool; it will not provide medical diagnoses, treatment recommendations, or emergency response protocols.

- **iOS Deployment:**
  While built on a cross-platform framework, immediate iOS deployment and App Store submission are excluded due to licensing constraints.

5. REQUIREMENTS ANALYSIS

This section outlines the functional and non-functional requirements for the updated Client-Server architecture. The transition from a rule-based offline tool to a machine learning-driven system introduces new cloud dependencies, requiring a strict separation of client-side edge operations (UI, image capture, OCR extraction) and server-side operations (NLP classification, semantic mapping, database queries).

5.1. Functional Requirements

- **FR-01 : Edge-Based Image Capture and Preprocessing**

  The Flutter mobile application must provide a dedicated camera interface equipped with a focus guide and low-light illumination controls. It must capture high-resolution images of food packaging and apply local mathematical preprocessing (grayscale conversion, contrast normalization, and noise reduction) to optimize the label for text extraction.

- **FR-02 : Edge Optical Character Recognition (OCR)**

  The client application must utilize an embedded OCR engine (Google ML Kit) to extract unstructured, noisy text from the preprocessed image entirely on the device. The system must format this raw text into a structured JSON payload suitable for transmission to the cloud backend.

- **FR-03 : Secure Client-Server Communication**

  The mobile client must transmit the extracted OCR text payload, along with the active user's profile ID, to the FastAPI cloud backend via a secure, authenticated RESTful API endpoint (HTTPS/TLS).

- **FR-04 : Cloud-Based NLP Classification**

  The FastAPI backend must receive the noisy OCR string and process it through a custom Natural Language Processing pipeline built with Python, spaCy, and scikit-learn. The model must clean, tokenize, and accurately classify the unstructured, misspelled text into standardized, recognized ingredient entities.

- **FR-05 : Relational Semantic Translation**

  The backend must query the PostgreSQL relational knowledge base using the classified ingredient entities. It must successfully map complex scientific terms, alternative naming

conventions, and E-numbers to their corresponding primary allergen groups (e.g., mapping "E322" and "lecithin" to soy or egg).

- **FR-06 : Personalized Allergen Evaluation**

  The backend logic must cross-reference the semantically mapped allergens against the specific dietary restrictions defined in the transmitted user profile. The system must calculate a definitive safety status (Safe, Caution, or Unsafe) based on this evaluation.

- **FR-07 : Result Visualization and Additive Education**

  The mobile client must parse the evaluated JSON payload returned by the server and render the results through an intuitive, color-coded interface. It must display explicit visual warnings for detected allergens and provide plain-language, educational explanations for identified artificial additives.

- **FR-08 : Multi-Profile Dietary Management**

  The system must allow users to securely register accounts and maintain up to five distinct dietary profiles (e.g., individual profiles for different family members). Users must be able to seamlessly switch between these profiles on the client side to evaluate a single scanned product against different dietary restrictions without requiring a physical re-scan.

- **FR-09 : Scan History and Data Persistence**

  The backend must log successful scans, securely storing the timestamp, predicted ingredients, and safety evaluation results in the PostgreSQL database. The mobile client must allow users to retrieve, filter, and view this historical data for future shopping reference.

5.2. Non-Functional Requirements

- **NFR-01 : Backend Latency and Inference Speed**

  The system must minimize the latency introduced by the Client-Server architecture. The end-to-end network process—from the moment the client transmits the OCR payload to the moment it receives the evaluated safety response—must complete in under 3.0 seconds under standard 4G/LTE network conditions. The FastAPI backend and scikit-learn inference pipeline must be highly optimized for rapid execution.

- **NFR-02 : Machine Learning Accuracy and Confidence Scoring**

  The NLP classification model must achieve a minimum F1-score of 0.85 on the validation dataset to ensure reliable ingredient identification from highly degraded OCR text. The backend must calculate and return a confidence score alongside its predictions. If the classification confidence falls below 75%, the system must trigger a "low confidence" prompt on the client UI, advising the user to manually review the physical label.

- **NFR-03 : Scalability and Concurrency**

  The backend infrastructure, containerized via Docker, must be capable of handling at least 100 concurrent user requests without degrading the sub-3-second response time threshold. The PostgreSQL database connection pool must be configured to prevent query bottlenecking during peak usage hours.

- **NFR-04 : Security and Data Privacy**

  All API communications between the Flutter client and FastAPI server must be encrypted in transit using TLS 1.2 or higher. User passwords must be heavily hashed using bcrypt, and session management must utilize JSON Web Tokens (JWT) with strict expiration windows. Sensitive personal dietary profiles and scan histories must be logically isolated within the relational database to prevent unauthorized cross-tenant data access.

- **NFR-05 : Maintainability and Model Retraining**

  The backend architecture must support modular, zero-downtime deployments to allow for seamless updates to the ML model weights and the PostgreSQL knowledge base. The data architecture must be structured so that newly reported OCR failure cases can be easily extracted, annotated, and fed back into the training pipeline for continuous, iterative model improvement.

- **NFR-06 : High Availability and Fallback**

  The cloud-hosted FastAPI backend and PostgreSQL database should be deployed with automated health checks and restart policies to target a 99.9% uptime. In the event of a total network failure, the mobile client must gracefully handle the timeout by alerting the user that an active internet connection is required for AI processing, rather than crashing unexpectedly.

6. PROPOSED SOLUTION

6.1. Overall Solution Concept (Client-Server Shift)

The proposed solution transitions from a constrained, on-device rule-based application to a scalable Client-Server architecture powered by machine learning. The core concept divides computational responsibilities to optimize both mobile hardware constraints and cloud processing power. The Flutter mobile client acts as an intelligent edge node, responsible solely for image acquisition, image preprocessing, and raw Optical Character Recognition (OCR) text extraction using Google ML Kit. The heavy computational burden of natural language understanding is offloaded to a Python-based FastAPI cloud backend. This backend hosts a custom-trained machine learning model that classifies the noisy, unstructured OCR text into definitive ingredient entities. Finally, a PostgreSQL relational knowledge base semantically translates these predicted chemical names and E-numbers into actionable, personalized allergen warnings based on the user's active dietary profile.

6.2. Major Features

- **Edge-Based Text Extraction**

  Real-time image capture and raw text extraction using mobile-optimized OCR, minimizing the payload size transmitted over the network.

- **ML-Driven Ingredient Classification**

  A cloud-hosted NLP model that intelligently identifies actual ingredients from a messy block of extracted OCR text, effectively ignoring marketing copy, nutritional tables, and manufacturer typos.

- **Semantic Translation and E-Number Mapping**

  A comprehensive PostgreSQL knowledge base that maps scientifically complex names (e.g., "sodium caseinate") and E-numbers (e.g., "E322") to their foundational allergen groups (milk, soy, etc.) and plain-language definitions.

- **Multi-Profile Personalization**

  The ability to store multiple distinct user profiles (e.g., different family members) and evaluate a single scanned product against any selected profile dynamically.

- **Deterministic Safety Visualization**

  Immediate, color-coded visual feedback categorizing a scanned product as Safe (Green), Caution/Additive (Yellow), or Danger (Red) based on strict cross-referencing between the ML model's output and the active user profile.

6.3. System Workflow (Edge OCR to Cloud NLP)

6.3. System Workflow (Edge OCR to Cloud NLP)

The system operates through a strictly ordered pipeline integrating edge processing, generative AI, and semantic vector matching:

1. **Image Capture and Preprocessing**

   The user captures a food label using the Flutter client. The app applies localized mathematical filters to enhance text legibility.

2. **Edge OCR Extraction**

   Google ML Kit processes the enhanced image directly on the device, extracting a single, unstructured string of raw text.

3. **Payload Transmission**

   The Flutter client constructs a JSON payload containing the raw OCR string and the active user's profile ID, transmitting it to the FastAPI backend.

4. **NLP/LLM Interpretation and Structured Extraction**

   The FastAPI server routes the text through a hosted LLM (e.g., OpenAI or Gemini) or custom NLP pipeline. This stage cleans OCR noise and converts the raw string into a structured list of individual ingredient candidates.

5. **Multi-Level Knowledge Base Matching**

   The server compares the candidates against the PostgreSQL Knowledge Base using a three-level strategy:

   - **Level 1 (Exact Matching):**

     If the normalized OCR term directly matches a known canonical name or alias in the database, the record is retrieved.

- **Level 2 (Semantic Matching):**
  If exact matching fails, an embedding model converts the term into a numerical vector and searches the pgvector index for semantically similar entries (e.g., matching "whey concentrate" to "whey protein").

- **Level 3 (Verification):**
  Vector similarity results are evaluated against a predefined threshold to retrieve the verified Knowledge Base relationship.

6. **Allergen Engine Processing**
   The Allergen Engine (a deterministic application logic layer) maps the retrieved ingredient entities and verified relationships (e.g., "whey protein" → "milk-derived") to specific allergen categories and additive definitions.

7. **Personalized Safety Evaluation and Result Generation**
   The system calculates the intersection between the detected product allergens and the selected user profile's dietary restrictions. FastAPI returns a JSON safety report, which the Flutter app renders as a color-coded UI warning with plain-language LLM-generated explanations.

6.4. User Interaction

The application is designed for rapid, low-friction interaction within a retail environment. The primary user flow requires only three interactions: opening the scanner, ensuring the correct family profile is active via a dropdown toggle, and capturing the physical label. In under three seconds, the UI transitions to a results dashboard. To accommodate non-technical users, complex chemical classifications are hidden behind intuitive color-coded banners. Users seeking deeper understanding can tap individual ingredient cards to read plain-language translations of artificial additives and preservatives.

6.5. Major Inputs and Outputs

- **Major Inputs**
  High-resolution digital photographs of physical food packaging, user-defined dietary profiles (boolean selections of nine major allergens), and manual text queries for the search fallback.

- **Major Outputs**

Structured JSON arrays of classified ingredient entities, deterministic safety status enumerations (Safe/Caution/Unsafe), confidence scores from the ML prediction model, and localized scan history logs saved to the database.

6.6. External Systems and Services

To support the Client-Server architecture, the system integrates several external services. Google ML Kit is utilized as the primary on-device OCR engine. The cloud backend is hosted on a scalable infrastructure provider (such as Render or AWS) running Python 3.10+ and the FastAPI framework. The relational knowledge base and user data are managed by a managed cloud PostgreSQL instance. Authentication and secure routing are handled via JSON Web Tokens (JWT) generated by the backend.

6.7. Integration Between Major Components

Integration is achieved through stateless RESTful API contracts. The mobile frontend (Developer A) communicates with the backend (Developer D) exclusively via standardized HTTP endpoints. The FastAPI backend serves as the central orchestrator, passing text data to the loaded scikit-learn ML model (Developer B) for inference, and subsequently constructing dynamic SQL queries to retrieve semantic mappings from the PostgreSQL database (Developer C). This modular integration ensures that the NLP model or database schema can be updated seamlessly on the server without requiring a forced app update for the mobile client.

7. SYSTEM ARCHITECTURE AND TECHNICAL DESIGN

7.1. High-Level Client-Server System Architecture

The updated system architecture completely deprecates the monolithic offline approach in favor of a distributed Client-Server model. This architecture is divided into three primary tiers: the Edge Client Tier, the Application Backend Tier, and the Data Persistence Tier. The Edge Client Tier (Flutter) is responsible for hardware interfacing and localized OCR extraction. The Application Backend Tier (FastAPI) serves as the orchestration layer, hosting the custom natural language processing (NLP) model for semantic classification. The Data Persistence Tier (PostgreSQL) manages the relational mapping of chemical components to dietary constraints. This decoupling ensures that the computationally heavy ML inference and complex database joins are executed on scalable cloud infrastructure, rather than constrained mobile hardware.

7.2. Major Components and Modules

- **Mobile Frontend Module**

  Developed in Flutter by Developer A (Mekala), this module manages the user interface, camera lifecycle, state management, and the visual rendering of the final safety report.

- **Edge OCR Module**

  Managed by Developer D (Dilshan), this component integrates Google ML Kit directly onto the device to handle localized image preprocessing and raw text extraction, acting as the primary data ingestion point.

- **NLP Classification Engine**

  Engineered by Developer B (Rashmika), this Python-based module utilizes spaCy for tokenization and text cleaning, and a trained scikit-learn classification model to predict ground-truth ingredient entities from the noisy OCR strings.

- **Backend API and Routing Module**

  Built by Developer D (Dilshan), the FastAPI framework handles secure HTTP routing, payload validation via Pydantic, JWT authentication, and coordinates the flow of data between the ML engine and the database.

- **Knowledge Base and Mapping Module**

  Architected by Developer C (Deemantha), this component leverages a highly normalized PostgreSQL database to handle complex semantic mapping, translating the ML engine's predicted entities (like E-numbers and chemical names) into overarching major allergen groups.

7.3. Interfaces (RESTful APIs)

Communication between the edge client and the cloud backend is strictly governed by a RESTful API architecture. Data payloads are serialized into structured JSON. The frontend communicates with the backend exclusively over HTTPS port 443 utilizing TLS 1.2+ encryption. The interfaces are designed to be stateless, meaning every inference request from the mobile client must include both the raw OCR text array and the active user's profile ID to generate an accurate, individualized safety response.

7.4. Data Flow (Mobile to FastAPI to PostgreSQL)

The operational data flow follows a strict sequential pipeline:

1. The Flutter client captures an image, and Google ML Kit extracts a noisy text string.
2. The mobile client wraps this text and the user profile ID into an HTTP POST request and sends it to the FastAPI endpoint.
3. FastAPI validates the payload and passes the raw text to the NLP Classification Engine.
4. The spaCy pipeline tokenizes the string, and the scikit-learn model performs inference, outputting a cleaned array of predicted ingredient entities with associated confidence scores.
5. FastAPI takes these predicted entities and executes a SQL query against the PostgreSQL database.
6. The database maps the entities to their specific allergen classifications and additive categories, returning the relational data to FastAPI.
7. FastAPI evaluates the detected allergens against the provided user profile ID, constructs a final JSON safety report, and returns it to the Flutter client for rendering.

7.5. Cloud Database Schema (PostgreSQL)

The transition to PostgreSQL allows for complex relational querying essential for semantic translation. The schema includes the following primary tables:

- **Users**

  Stores authentication credentials (hashed passwords) and account metadata.

Field Name	Data Type	Description	Constraints/Keys
user_id	UUID	Unique identifier for the user.	Primary Key
email_address	VARCHAR	User's email for login and recovery.	Unique, Not Null

password_hash	VARCHAR	Securely hashed user password via bcrypt.	Not Null
account_creation_date	TIMESTAMP	Timestamp of account registration.	Default: CURRENT_TIMESTAMP

Table 2: Users Database Schema

- **Profiles**

Linked to Users via a foreign key, storing specific allergen Boolean flags for different family members.

Field Name	Data Type	Description	Constraints/Keys
profile_id	UUID	Unique identifier for the profile.	Primary Key
user_id	UUID	Links the profile to the account owner.	Foreign Key (Users)
profile_name	VARCHAR	Display name (e.g., "Child Profile").	Not Null
active_allergens	JSONB	Array of selected FDA allergen IDs to avoid.	Not Null

Table 3: Profiles Database Schema

- **Ingredients**

The core dictionary of standardized chemical and food names predicted by the ML model.

Field Name	Data Type	Description	Constraints/Keys
ingredient_id	UUID	Unique identifier for the ingredient entity.	Primary Key
standard_name	VARCHAR	Ground-truth name predicted by the NLP model.	Unique, Not Null
alternative_names	JSONB	Array of chemical synonyms to assist mapping.	
explanation	TEXT	Plain-language educational explanation.	

Table 4: Ingredients Database Schema

- **Allergens**

The nine major FDA-recognized allergen groups.

Field Name	Data Type	Description	Constraints/Keys
allergen_id	INT	Unique identifier for the allergen.	Primary Key
allergen_name	VARCHAR	Name of the major allergen (e.g., Milk, Peanut).	Unique, Not Null
description	TEXT	General medical information about the allergen.	

Table 5: Ingredients Database Schema

- **Ingredient_Allergen_Mapping**

  A junction table facilitating many-to-many relationships (e.g., "whey" and "casein" both mapping to "milk").

Field Name	Data Type	Description	Constraints/Keys
map_id	UUID	Unique identifier for the mapping record.	Primary Key
ingredient_id	UUID	Links to the standardized ingredient entity.	Foreign Key (Ingredients)
allergen_id	INT	Links to the associated major allergen group.	Foreign Key (Allergens)

Table 6: Ingradient_Allergen_Mapping Database Schema

- **Additives**

  Stores E-numbers and plain-language functional descriptions.

Field Name	Data Type	Description	Constraints/Keys
additive_id	UUID	Unique identifier for the additive.	Primary Key
e_number	VARCHAR	Standardized European additive code (e.g., "E322").	Unique, Not Null
standard_name	VARCHAR	Standard chemical name of the additive.	Not Null
category	VARCHAR	Function category (e.g., Preservative, Colorant).	
explanation	TEXT	Plain-language educational explanation.	

Table 7: Additives Database Schema

- **Inference_Logs**

  A feedback loop table that archives noisy OCR strings alongside the model's final predictions, allowing Developer B to continuously extract new training data for future model optimization.

Field Name	Data Type	Description	Constraints/Keys
log_id	UUID	Unique identifier for the inference event.	Primary Key
user_id	UUID	Links to the user who initiated the scan.	Foreign Key (Users)
scan_timestamp	TIMESTAMP	Date and time the server processed the scan.	Default: CURRENT_TIMESTAMP
raw_ocr_text	TEXT	The noisy string extracted by Google ML Kit at the edge.	Not Null
predicted_entities	JSONB	The cleaned ingredient list predicted by the NLP model.	Not Null

confidence_score	FLOAT	The F1-derived confidence probability of the model's classification.	Not Null
safety_result	VARCHAR	Overall deterministic outcome (Safe, Caution, Warning).	Not Null

Table 8: Inference Log Database Schema

- **Ingredient Embeddings**

  Stores numerical vector representations of canonical ingredient descriptions and aliases to support semantic retrieval via pgvector

Field Name	Data Type	Description	Constraints/Keys
embedding_id	UUID	Unique identifier for the vector record.	Primary Key
ingredient_id	UUID	Links to the standardized ingredient entity.	Foreign Key (Ingredients)
embedding_vector	VECTOR	The numerical semantic embedding of the term.	Not Null

7.6. API and Service Interfaces

The FastAPI backend exposes several core endpoints documented via an auto-generated Swagger/OpenAPI interface:

- POST /api/v1/auth/login: Issues JWT access and refresh tokens.
- GET /api/v1/profiles: Retrieves all dietary profiles associated with the authenticated user.
- POST /api/v1/analyze: The primary inference endpoint. Accepts OCR text and profile ID; returns predicted ingredients, detected allergens, confidence scores, and the final safety determination.
- GET /api/v1/history: Retrieves paginated historical scan logs for the active user.

7.7. Client, Server, and Device Relationships

The system follows a "thin client, thick server" paradigm for its intelligence layer. The mobile device acts strictly as a data collection tool (camera/OCR) and presentation layer. It possesses no inherent knowledge of food chemistry or allergen mapping. The server acts as the authoritative intelligence node. This relationship ensures that the ML model and the chemical knowledge base can be updated, refined, and scaled by the development team without requiring end-users to repeatedly download application updates from the app store.

7.8. Deployment Environment

The backend ecosystem relies on containerization for consistent deployment. The FastAPI application and the serialized scikit-learn model weights are bundled into a Docker container. This container is deployed to a managed cloud hosting environment (e.g., Render or AWS App Runner). The PostgreSQL database is hosted on a managed cloud relational database service to ensure high availability and automated backups. The mobile application is compiled into an Android APK for distribution. A continuous integration/continuous deployment (CI/CD) pipeline via GitHub Actions automates backend testing and deployment upon code merges.

7.9. Relevant Diagrams

The final technical documentation will feature the following newly designed architectural diagrams to reflect the Client-Server shift:

1. **System Architecture Diagram**

   Visualizing the physical separation between the Flutter client device, the FastAPI cloud runtime, and the PostgreSQL database cluster.

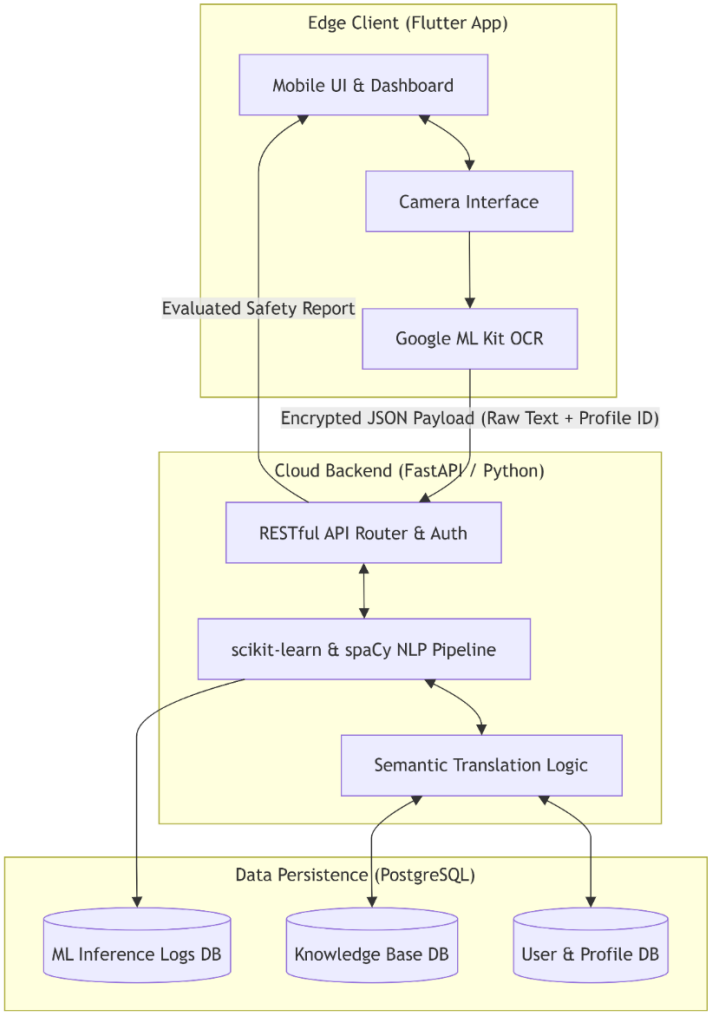


Figure 1: High Level System Architecture

2. Data Flow Diagram (Level 0 and 1)

Detailing the transformation of raw OCR text through the NLP inference model and the semantic translation layer.

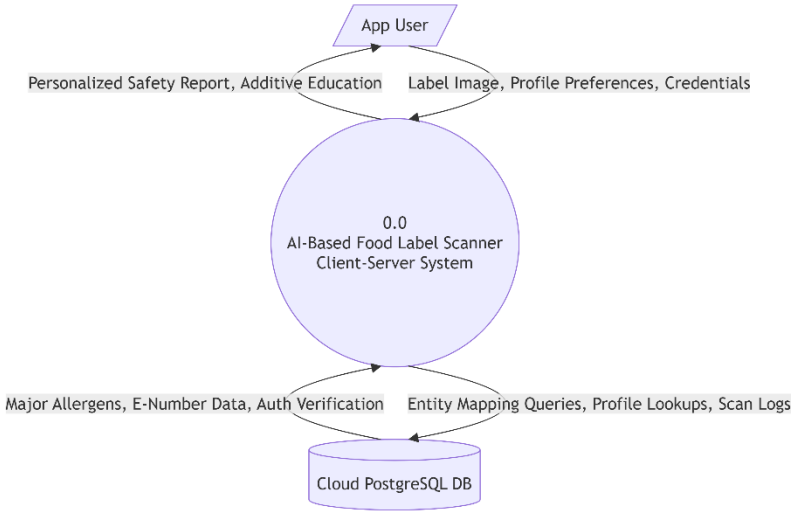


Figure 2: DFD Level 0

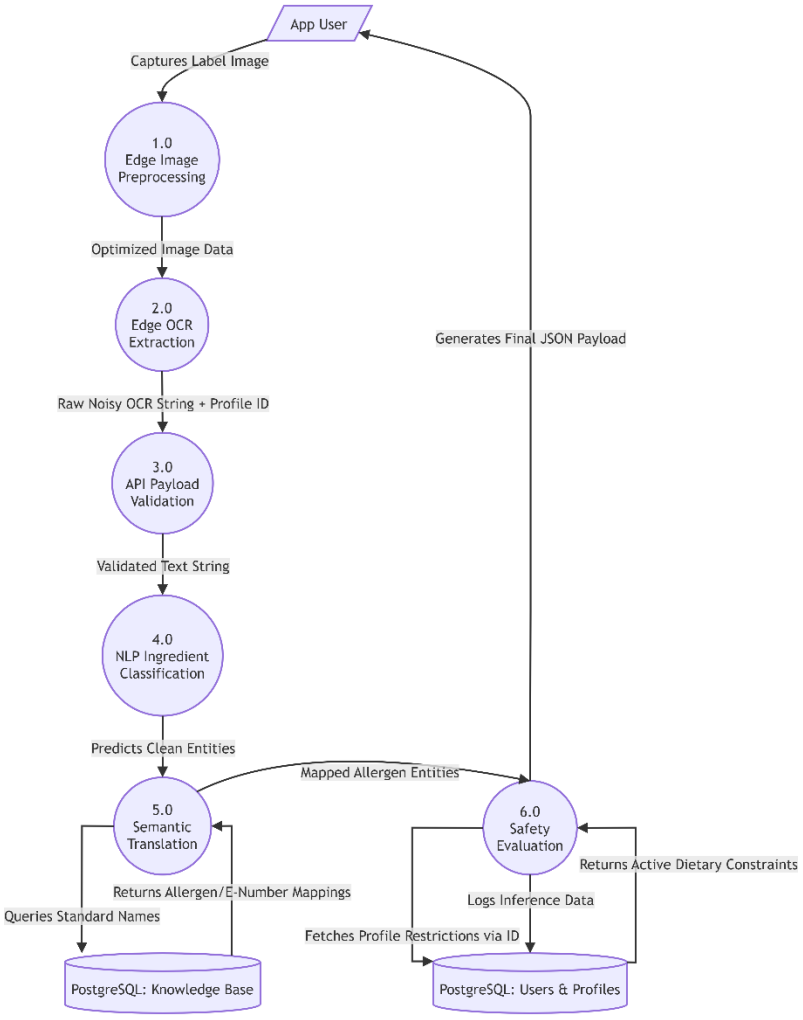


Figure 3: DFD Level 01

3. Entity-Relationship (ER) Diagram

Mapping the complex SQL joins between the Users, Profiles, Ingredients, and Allergens tables.

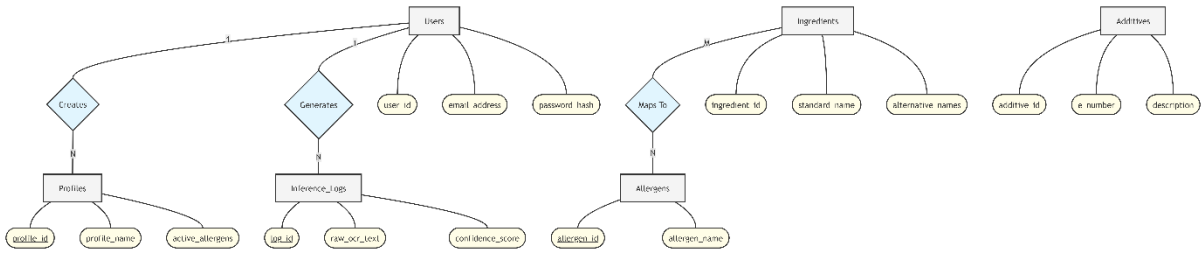


Figure 4: ER Diagram

4. Sequence Diagram

Outlining the synchronous chronological steps and API handshakes required for a single label scan.

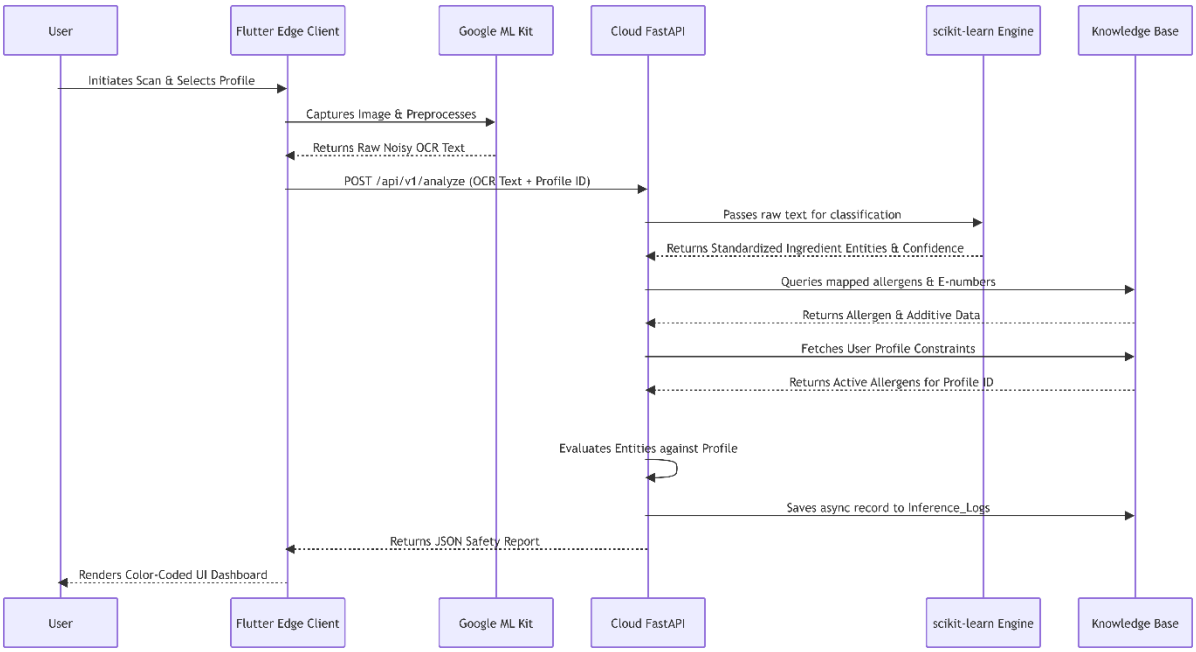


Figure 5: Sequence Diagram

5. Use Case Diagram

The use-case diagram maps the primary interactions between the app user and the client-server system, illustrating how localized edge actions like image capture trigger cloud-based machine learning classifications to generate personalized safety reports.

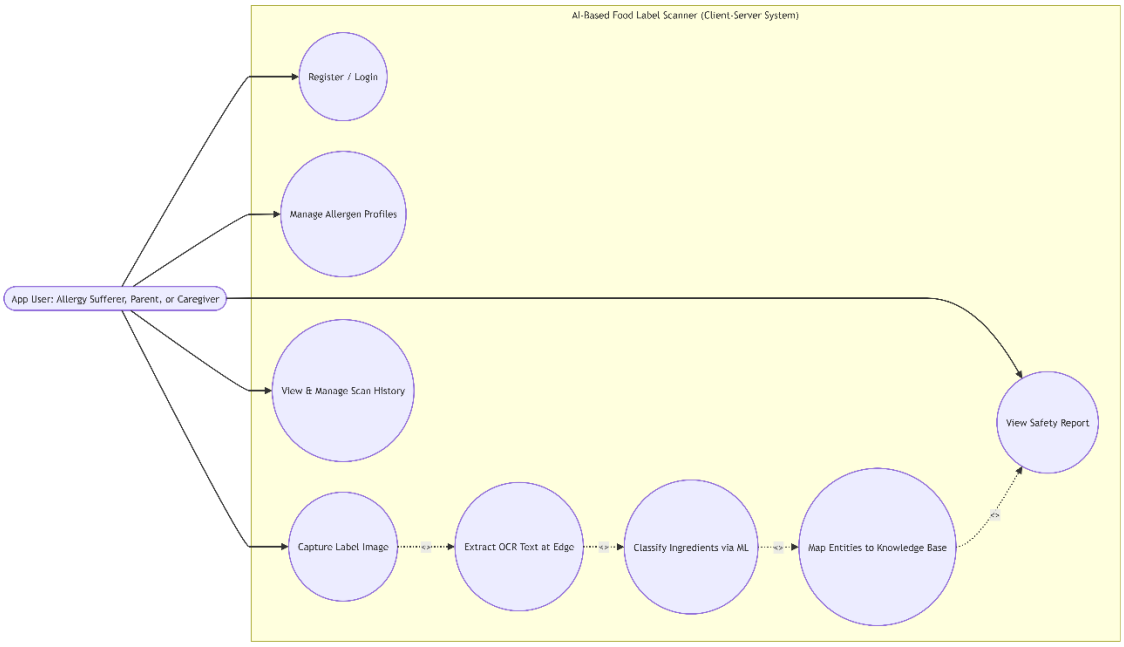


Figure 6: Use Case Diagram

6. Mobile UI Wireframes

Displaying the updated frontend interface, including the profile switching toggle and the network loading states required for cloud inference.

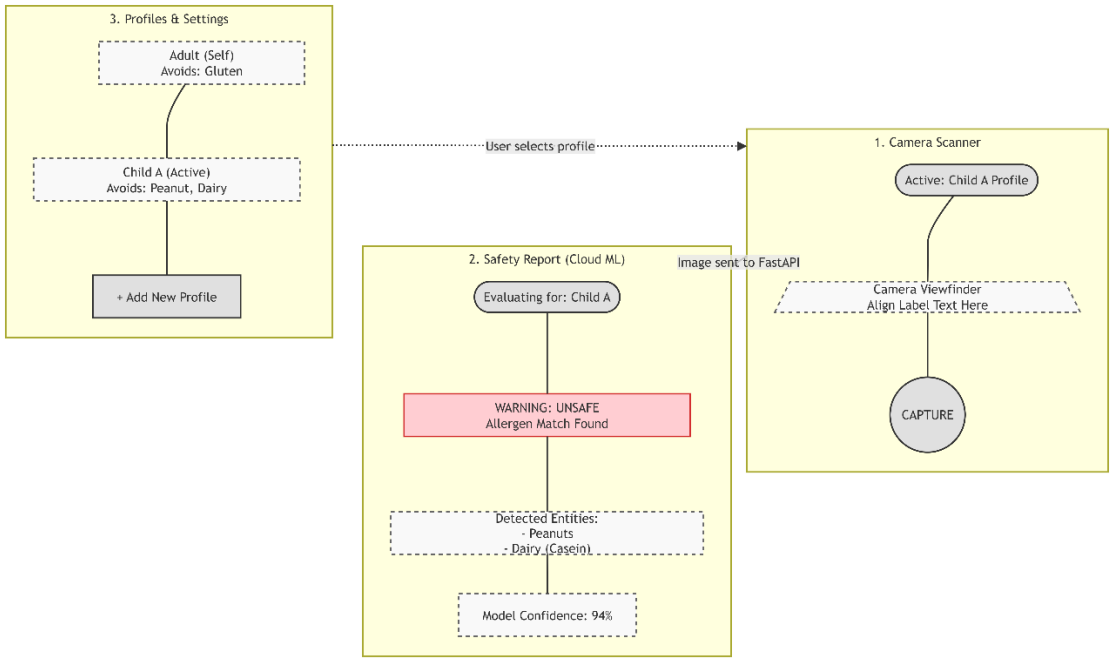


Figure 7: Mobile UI Wireframe

8. TECHNOLOGY SELECTION AND JUSTIFICATION

8.1. Programming Languages

- **Dart**

  Selected as the primary language for the mobile client. Dart compiles directly to native ARM code, which is critical for maintaining high performance during real-time camera rendering and edge-based image preprocessing. Its strong typing and seamless integration with the Flutter framework ensure a responsive, low-latency user interface.

- **Python (3.10+)**

  Selected as the exclusive language for the cloud backend and machine learning pipeline. Python dominates the AI and data science ecosystem, offering unparalleled native support for the required Natural Language Processing (NLP) libraries. It allows the team to unify the machine learning training scripts and the production server routing within a single language environment.

8.2. Frameworks and Libraries

- **Flutter (Mobile Framework):**

  Utilized to build the cross-platform client application. Flutter's direct hardware interoperability allows for efficient integration with native camera plugins and on-device processing workflows, eliminating the latency introduced by JavaScript bridges found in alternatives like React Native.

- **Google ML Kit (Edge OCR):**

  Chosen for on-device raw text extraction. By performing OCR at the edge, the system reduces the network payload size (sending text strings rather than high-resolution images) and ensures immediate data capture without server-side image processing bottlenecks.

- **FastAPI (Backend Framework):**

  Selected to replace the offline logic tier. FastAPI is a high-performance, asynchronous Python web framework optimized for building RESTful APIs. It handles concurrent I/O operations efficiently, making it the ideal orchestration layer for serving machine learning inferences and routing database queries under load.

- **scikit-learn & spaCy (AI/NLP Pipeline):**

  These libraries replace the rejected rule-based keyword matching approach. spaCy handles advanced NLP tasks such as rapid tokenization, text cleaning, and Named Entity Recognition (NER) on the noisy OCR strings. scikit-learn is utilized to train and deploy the supervised classification model, enabling the system to accurately predict ground-truth ingredient entities from misspelled or fragmented inputs.

- **OpenAI / Gemini API (LLM Integration):**

  Introduced to handle complex interpretation of abbreviated or highly degraded OCR text, extracting structured ingredient lists (JSON) and generating user-friendly plain-language explanations. This reduces the hallucination risk by isolating the LLM's role to text processing, while allergen logic remains governed by the deterministic database.

- **Embedding Models (e.g., OpenAI embeddings):**

  Utilized to convert recognized ingredient phrases into numerical semantic vectors. This allows the system to understand that "whey concentrate" and "whey protein" are conceptually linked, even when character-based matching fails.

8.3. Database and Storage

- **PostgreSQL (Relational Knowledge Base):**

  The architecture shifts from a local SQLite file to a managed cloud PostgreSQL instance. PostgreSQL provides the rigid structural integrity and advanced relational querying capabilities required to map thousands of predicted chemical entities and E-numbers to overarching major allergen groups. It also safely centralizes user credentials, multi-tenant dietary profiles, and ML inference feedback logs.

- **PostgreSQL with pgvector:**

  The architecture utilizes a managed cloud PostgreSQL instance as the structured source of truth for all verified ingredient relationships and user profiles. The `pgvector` extension is integrated directly into the database to serve as the retrieval mechanism for semantic similarity searches, eliminating the strict necessity for a separate, standalone vector database service like Pinecone.

8.4. Platforms and Development Tools

- **Jupyter Notebooks:**

  Introduced specifically for the new Machine Learning pipeline. Jupyter provides an interactive environment for data cleaning, exploratory data analysis (EDA) on the noisy OCR dataset, and iterative model training before exporting the final model weights.

- **Android Studio & Visual Studio Code:**

  Android Studio provides the necessary emulators and visual layout inspection tools for Flutter. Visual Studio Code serves as the lightweight environment for Python backend development and API routing.

- **Figma:**

  Utilized for real-time, collaborative UI/UX wireframing to ensure clear visual state propagation for the color-coded safety warnings prior to mobile implementation.

- **Postman:**

  Integrated for comprehensive testing of the FastAPI RESTful endpoints, allowing the team to simulate client payloads and validate JSON responses without requiring the compiled mobile app.

8.5. Cloud and Deployment Technologies

- **Docker:**

  The FastAPI application, NLP dependencies, and serialized scikit-learn model weights are containerized using Docker. This ensures the backend runs consistently across local development environments and the production server, eliminating deployment configuration errors.

- **Render / Railway (Cloud Hosting):**

  Selected for backend deployment due to native GitHub integration and automated CI/CD pipelines. These platforms facilitate rapid provisioning of the containerized Python services and managed PostgreSQL databases, avoiding the steep learning curves of enterprise AWS infrastructure while satisfying academic project requirements.

- **GitHub Actions:**

  Implemented to construct automated Continuous Integration and Continuous Deployment (CI/CD) pipelines directly within the repository. Workflows are configured to execute Python linting and testing protocols automatically upon pull request submissions.

9. TECHNICAL DEPTH AND CORE ENGINEERING WORK

This section details the primary engineering challenges and the technical implementations required to transition the system from a theoretical rule-based prototype to a robust, machine learning-driven architecture capable of handling real-world data imperfections.

9.1. ML Dataset Creation and Annotation (Handling OCR Noise)

The most significant barrier to accurate allergen detection is the high noise variance in OCR extraction caused by curved packaging, reflective wrappers, and poor lighting. To address this, Developer B (Rashmika) leads a dedicated dataset creation phase. The team will collect hundreds of physical food labels from local Sri Lankan supermarkets, capturing them under various environmental conditions to generate raw, noisy text via Google ML Kit.

This noisy text forms the independent variables (features) of the dataset. The text will be manually annotated to create ground-truth labels (target variables), mapping the fragmented or misspelled strings to standardized ingredient entities. For example, noisy inputs such as "wh ey c0ncentrat", "case1n", and "m!lk" will all be annotated with the clean target entity "casein" or "whey". This supervised dataset will be structured into a tabular format (CSV/JSON), establishing a direct mathematical relationship between OCR anomalies and actual food chemistry terms, forming the foundation for the NLP model.

9.2. NLP Classification Model Training and Optimization

To process the unstructured OCR strings, the system relies on a custom Natural Language Processing pipeline rather than static regular expressions. Utilizing Python, Developer B will implement text preprocessing using spaCy. The spaCy pipeline will perform tokenization, lemmatization, and stop-word removal to normalize the raw input strings.

Following preprocessing, the text data will be vectorized using Term Frequency-Inverse Document Frequency (TF-IDF) or word embeddings to convert the string data into numerical feature arrays. These arrays will be fed into a scikit-learn classification algorithm (e.g., Support Vector Machine (SVM) or a Random Forest Classifier) designed for multi-class text categorization. The model will be trained to predict the presence of specific ground-truth ingredient entities from the noisy vectors.

Optimization will focus heavily on the F1-score to ensure a balance between precision (minimizing false positives) and recall (minimizing false negatives, which are life-threatening in the context of food

allergies). Hyperparameter tuning will be conducted using GridSearchCV, and the final validated model weights will be serialized using joblib for deployment.

9.3. Semantic Translation and Allergen Engine Implementation

Once the OCR text is interpreted into structured ingredient candidates, the system must translate these scientific terms into actionable health data. Developer C will engineer the Knowledge Base and Allergen Engine, operating across FastAPI and PostgreSQL.

To solve semantic ambiguity, the backend implements a Multi-Level Matching Strategy:

1. Exact Matching: Fast database queries attempt to find direct matches for normalized text.
2. Semantic Matching: For unrecognized or heavily misspelled terms, the system generates an embedding via an external API and queries the `ingredient_embeddings` table using pgvector. This ensures that obscure terminology (e.g., "WPC") successfully resolves to its foundational entity ("whey protein concentrate").
3. Verification: The system validates the similarity threshold to prevent false positives.

Following successful entity retrieval, the data passes to the Allergen Engine. This deterministic application logic layer utilizes complex SQL JOIN operations across the `Ingredient_Allergen_Mapping` table. It converts the recognized ingredients and verified relationships into overarching allergen categories (e.g., translating "Sodium Caseinate" into a "Milk" warning) before evaluating those specific allergens against the user's active profile.

9.4. FastAPI Backend and Inference Integration

The orchestration of the entire pipeline is managed by Developer D (Dilshan) through the FastAPI backend. This component serves as the critical bridge between the mobile client and the machine learning model.

Upon server startup, FastAPI will load the serialized scikit-learn model and spaCy NLP pipeline into active memory to eliminate cold-start latency during individual API requests. When the client submits an HTTP POST request containing the noisy OCR payload, FastAPI utilizes Pydantic models to strictly validate the incoming JSON structure.

The validated text is passed to the in-memory ML model for entity prediction. The backend then asynchronously executes the semantic translation queries against the PostgreSQL database using asyncpg or SQLAlchemy, ensuring non-blocking database operations. After evaluating the returned allergens against the user's specific profile constraints, FastAPI constructs the final color-coded safety

response. Crucially, before returning the HTTP 200 response to the client, the backend asynchronously writes the original noisy text, the model's predictions, and the confidence scores into the Inference_Logs table. This creates an automated feedback loop, allowing the team to harvest edge-case failures for future dataset expansion and model retraining.

10.DATA, SECURITY, PRIVACY AND ETHICAL CONSIDERATIONS

10.1. Types and Sources of Training Data

The transition to a machine learning-driven architecture requires the management and protection of several distinct data categories:

- **Training Data:**
  A proprietary dataset consisting of noisy OCR-extracted text mapped to ground-truth ingredient entities, collected via manual annotation of physical Sri Lankan food labels.

- **Account Data:**
  User email addresses and hashed passwords required for system authentication.

- **Health Data:**
  User-defined allergen profiles and dietary restrictions, which dictate the semantic translation logic and safety evaluations.

- **Knowledge Base Data:**
  Curated medical and regulatory information mapping standardized chemical ingredients to major allergen groups.

10.2. Personal and Sensitive Information

The application processes highly sensitive personal data. Allergen profiles transcend basic dietary preferences and represent critical medical information. Unauthorized access to this data could expose a user's underlying medical conditions to malicious exploitation. Furthermore, the historical logs of scanned products can inadvertently reveal personal shopping habits, socio-economic indicators, and household dietary patterns. Therefore, the system architecture must treat this information with the rigorous security standards applied to electronic medical records.

10.3. Authentication, Authorization, and Data Protection

To secure the Client-Server data pipeline, the FastAPI backend implements a robust JSON Web Token (JWT) authentication system. Passwords are cryptographically hashed using the bcrypt algorithm prior to storage in the PostgreSQL database.

Because data is now transmitted between the mobile edge and the cloud backend, all in-transit communications are strictly encrypted via HTTPS utilizing TLS 1.2 or higher protocols. Within the

cloud database, role-based access control (RBAC) enforces logical tenant isolation, ensuring that personal user profiles and scan histories are exclusively accessible to the authenticated account owner.

10.4. Ethical and Legal Considerations (Medical Disclaimer)

The AI-Based Food Label Scanner is engineered strictly as an informational decision-support tool and must never be classified or utilized as a medical diagnostic device. Machine learning models and OCR extraction engines are inherently probabilistic and subject to classification errors. Additionally, food manufacturers may alter product formulations or introduce cross-contamination without updating packaging immediately.

To mitigate legal and ethical liabilities, the mobile UI will present a mandatory medical disclaimer during the initial user onboarding flow. This disclaimer will explicitly state that the application cannot guarantee 100% safety, that users must continue to manually verify official packaging, and that individuals with severe, life-threatening allergies must consult medical professionals and carry emergency medication at all times. The disclaimer limits the liability of the developers in the event of an allergic reaction occurring despite using the application.

11.TESTING AND EVALUATION PLAN

To ensure the reliability, accuracy, and safety of the newly implemented Client-Server architecture and machine learning pipeline, a comprehensive testing strategy is mandated. This plan evaluates both traditional software engineering metrics and specific data science performance indicators.

11.1. Unit and Integration Testing

The testing lifecycle begins with isolated unit tests to validate individual component logic before full system integration.

- **Frontend (Flutter):**
  Developer A will utilize the `flutter test` framework to validate the UI state management, profile toggling logic, and the localized edge image preprocessing algorithms.

- **Backend (FastAPI):**
  Developer D will implement automated unit tests using `pytest` to verify the RESTful API routing, payload validation via Pydantic, and JWT authentication flows.

- **Integration Testing:**
  The team will conduct end-to-end integration tests to validate the data pipeline between the edge client and the cloud backend. This includes simulating HTTP POST requests containing raw OCR text and verifying that the FastAPI server correctly routes the data through the NLP engine, executes the PostgreSQL semantic joins, and returns the appropriately structured JSON safety report. Mocking will be used to simulate database connections during early integration phases.

11.2. Machine Learning Evaluation (F1-Score, Precision, Recall)

Because the system relies on an NLP classification model to predict ingredient entities from noisy OCR data, traditional software testing is insufficient. Developer B will rigorously evaluate the scikit-learn model using standard machine learning metrics on a hold-out test dataset (a 20% split of the manually annotated OCR dataset not used during training).

- **Precision:**
  Measures the proportion of positive identifications that were actually correct. High precision is required to minimize false positives, which could unnecessarily flag safe food as dangerous, leading to user frustration.

- **Recall:**

  Measures the proportion of actual positives that were identified correctly. High recall is absolutely critical for this application, as a false negative (failing to identify an allergen) could result in a severe medical emergency.

- **F1-Score:**

  The harmonic mean of precision and recall. The model must achieve a minimum F1-score of 0.85 to be considered viable for deployment.

- **Confusion Matrix:**

  A visual table will be generated during evaluation to identify specific edge-case misclassifications, such as the model confusing similar chemical strings (e.g., misclassifying "caseinate" due to heavy OCR distortion).

11.3. System and Performance Testing (Backend Latency)

The transition to a cloud-based architecture introduces network dependency, making latency a primary performance concern. Developer D will conduct rigorous load testing on the deployed FastAPI backend using tools such as Locust or Apache JMeter.

- **Latency Benchmarks:**

  The system will be evaluated under varying network conditions (3G, 4G, and Wi-Fi). The total round-trip time—from the client transmitting the OCR payload to rendering the final UI evaluation—must consistently remain under 3.0 seconds.

- **Concurrency Testing:**

  The FastAPI server and PostgreSQL connection pools will be subjected to stress testing with up to 100 concurrent virtual users to identify memory leaks, query bottlenecks, or CPU throttling during NLP inference.

11.4. User Acceptance Testing

User Acceptance Testing (UAT) serves as the final validation phase, ensuring the application meets the practical needs of its target demographic.

- **Methodology:**

  A minimum of five external volunteers, specifically comprising individuals with diagnosed food allergies or parents managing complex dietary households, will utilize the application in real-world retail environments for one week.

- **Objectives:**

  UAT will assess the usability of the camera interface, the intuitiveness of the color-coded safety warnings, and the efficiency of the new multi-profile switching feature.

- **Feedback Integration:**

  Volunteers will submit feedback via structured questionnaires. Any critical usability flaws or severe latency perceptions identified during UAT will be addressed prior to the final academic submission.

12.DEVELOPMENT PROCESS AND GITHUB PLAN

12.1. Development Workflow and Branching Strategy

The project will adopt an Agile development methodology utilizing two-week sprints. To manage the complexities of a Client-Server architecture and parallel machine learning development, the team will strictly adhere to the GitHub Flow branching strategy. The 'main' branch will serve as the stable, production-ready codebase. Active development will occur on isolated feature branches branching off from 'main'. Branch naming conventions will clearly reflect the specific architectural components being addressed, utilizing prefixes such as 'feature/flutter-ocr-edge', 'feature/spacy-nlp-pipeline', or 'feature/postgres-schema'.

12.2. Integration Approach and Code Review

Given the distributed nature of the system, an incremental integration approach is essential to prevent late-stage architectural failures. During the initial sprints, the mobile client (Developer A) will develop against mocked API responses, while the FastAPI backend (Developer D) integrates the NLP classification model (Developer B) and PostgreSQL database (Developer C) locally.

All code merges into the 'main' branch will require a formal Pull Request (PR). To ensure code quality, security, and system stability, every PR must be reviewed and approved by at least one other team member. Furthermore, PRs will be subjected to automated Continuous Integration (CI) checks via GitHub Actions. These automated pipelines will execute Python linting and backend unit tests (pytest) to verify that new ML weights or database queries do not break existing API contracts before allowing a merge.

12.3. GitHub Repository Details

The project will be managed utilizing two separate GitHub repositories to cleanly decouple the mobile edge client from the cloud infrastructure.

- **Frontend Repository:**
  Dedicated exclusively to the Flutter Dart application, managing UI components, state management, and the edge-based Google ML Kit OCR integrations.

- **Backend Repository:**
  Dedicated to the Python FastAPI API, the Jupyter notebooks used for ML dataset cleaning and training, the serialized scikit-learn model weights, and the Docker deployment configurations.

To satisfy academic evaluation criteria, all team members (Mekala, Rashmika, Deemantha, and Dilshan) must utilize their individually identifiable GitHub accounts and maintain a consistent history of meaningful commits throughout the development lifecycle across both repositories where applicable. A final bulk upload at the conclusion of the project will not be accepted as valid evidence of continuous engineering effort. Comprehensive documentation, including the RESTful API endpoint contracts and the relational database schema, will be actively maintained within the README files of their respective repositories.

13.PROJECT MANAGEMENT PLAN

To meet the rigorous evaluation standards of the Faculty of Computing at Sabaragamuwa University of Sri Lanka, the project management lifecycle utilizes professional Agile methodologies tailored to the complexities of distributed Client-Server development.

13.1. Project Management Tool (Jira)

The team will utilize Jira by Atlassian as the primary project management and issue-tracking tool. Jira's native support for Agile Scrum workflows enables the team to plan structured two-week sprints, track velocity via burndown charts, and manage interdependent tasks across the frontend and backend repositories. The project board will be organized into specific Epics reflecting the new architecture (e.g., "Edge OCR Pipeline," "ML Dataset & Training," "PostgreSQL Knowledge Base," "FastAPI Orchestration"). Individual tasks will be logged as User Stories, and bug reports discovered during integration testing will be assigned severity labels (Critical, Major, Minor) to prioritize resolution.

13.2. Development Phases (Incorporating Dataset Collection & Model Training)

The project is scheduled across a 15-week timeline, divided into seven distinct phases to accommodate the new machine learning data collection and training prerequisites.

Phase 1: Project Setup and UI Prototyping (Week 1)

Initialize the dual GitHub repositories (Frontend and Backend) with branching structures. Finalize the Figma UI/UX designs for the color-coded safety dashboard. Configure the development environments for Flutter, Python, and PostgreSQL.

Phase 2: Edge OCR Pipeline and Noisy Data Collection (Weeks 2-3)

Developer A (Mekala) builds the Flutter camera interface, while Developer D (Dilshan) integrates Google ML Kit for raw text extraction. Concurrently, Developer B (Rashmika) leads the physical data collection, scanning hundreds of real-world food labels to generate the noisy OCR dataset required for ML training.

Phase 3: ML Model Training and Database Schema Construction (Weeks 4-6)

Developer B cleans the noisy OCR dataset, annotates ground-truth entities, and trains the scikit-learn/spaCy classification model to achieve a high F1-score. Developer C (Deemantha) constructs the PostgreSQL database schema and populates the semantic mapping tables for allergens and E-numbers.

Phase 4: Client-Server API Integration (Weeks 7-9)

Developer D builds the FastAPI endpoints and integrates the trained ML model for live inference. Developer A integrates the Flutter frontend with the RESTful API. The system performs its first end-to-end cloud inference, converting an image to a semantic PostgreSQL query.

Phase 5: Multi-Profile Logic and Advanced UI (Weeks 10-11)

Implement the user authentication (JWT) and multi-profile dietary management logic. Developer D dockerizes the FastAPI backend for consistent deployment. Developer A refines the mobile UI to handle loading states and network timeout edge cases.

Phase 6: System Evaluation and User Acceptance Testing (Weeks 12-13)

Conduct rigorous evaluation of the ML model metrics (Precision, Recall) on a hold-out test set. Perform latency load testing on the FastAPI backend. Execute User Acceptance Testing (UAT) with external volunteers managing food allergies. Resolve all critical bugs.

Phase 7: Deployment and Academic Documentation (Weeks 14-15)

Deploy the Dockerized backend to a cloud hosting platform and finalize the production PostgreSQL instance. Compile the final signed Android APK. Complete all technical documentation, API contracts, and prepare the project viva presentation.

13.3. Major Activities, Milestones, and Module Responsibilities

The timeline is anchored by specific, verifiable milestones to ensure continuous progress:

- **Milestone 1:**
  Successful edge extraction of noisy OCR text via the Flutter mobile client.

- **Milestone 2:**
  Completion of the annotated training dataset and the serialization of a deployed scikit-learn model achieving an F1-score > 0.85.

- **Milestone 3:**
  Functional PostgreSQL knowledge base accurately translating chemical entities to major allergen groups.

- **Milestone 4:**

  End-to-end integration demonstrating a successful HTTP POST request yielding a personalized safety JSON response in under 3.0 seconds.

- **Milestone 5:**

  Final production deployment of the cloud infrastructure and mobile APK.

The module responsibilities are strictly defined to prevent overlap and ensure accountability:

- **Developer A (Mekala):**

  Mobile App, Scanner Interface, Dashboard UI, and Client-Side State Management.

- **Developer B (Rashmika):**

  ML Dataset Creation, Data Annotation, spaCy Tokenization, and scikit-learn Model Training.

- **Developer C (Deemantha):**

  PostgreSQL Schema Design, E-Number Recognition, and Semantic Allergen Detection.

- **Developer D (Dilshan):**

  FastAPI Backend Development, Google ML Kit Edge Integration, JWT Authentication, and Docker Deployment.

14.FEASIBILITY AND RISK ANALYSIS

14.1. Technical and Performance Risks

- **OCR Extraction Failure:**
  Curved packaging, poor lighting, or reflective wrappers may cause Google ML Kit to return severely distorted or incomplete text strings.

  ✓ Mitigation : The machine learning pipeline is specifically trained on distorted inputs to handle this noise. If the OCR string is completely empty or illegible, the Flutter UI will prompt the user to adjust the lighting and retake the photo, or provide a manual text-entry fallback.

- **Cloud Latency and Connectivity:**
  The shift from an offline application to a Client-Server architecture introduces a strict reliance on internet connectivity. A slow network could delay the safety report, degrading the user experience.

  ✓ Mitigation : The Flutter client will implement asynchronous loading states and timeout handlers. The FastAPI backend and PostgreSQL database will be hosted in the same cloud availability zone to minimize internal server latency.

14.2. Machine Learning and Data Risks

- **High False Negative Rate:**
  The most critical system risk is the NLP model failing to identify a dangerous allergen from a misspelled string, potentially causing a life-threatening medical event.

  ✓ Mitigation : The scikit-learn model will be strictly optimized and penalized for false negatives during the training phase. Evaluation will prioritize the Recall metric over Precision. Furthermore, the PostgreSQL semantic mapping ensures that once an entity is identified, its hidden allergens are deterministically flagged without relying on probability.

- **Dataset Bias:**

  Training the model strictly on limited local packaging may reduce the model's accuracy when users scan imported goods with different regulatory naming conventions.

  - ✓ Mitigation : The dataset collection phase will purposely target a diverse mix of local Sri Lankan products and imported goods. The Inference_Logs database table will continuously capture real-world edge-case failures to facilitate continuous model retraining.

14.3. Project Management and Schedule Risks

- **Scope Creep and Time Constraints:**

  Integrating an ML pipeline, a cloud API, and a mobile frontend within a 15-week academic timeline is highly ambitious and risks schedule overruns.

  - ✓ Mitigation : The team will enforce strict adherence to the Agile two-week sprint methodology managed via Jira. Extraneous features (such as barcode scanning or community recipe sharing) have been explicitly excluded from the system boundary to focus engineering efforts entirely on the core OCR and NLP classification flow.

14.4. Legal and Ethical Risks

- **Medical Liability:**

  Users might suffer allergic reactions if food manufacturers alter product formulations without updating the packaging, or if the model misclassifies a novel ingredient.

  - ✓ Mitigation : A mandatory medical disclaimer must be acknowledged during user registration. The application is strictly positioned as an informational decision-support tool, not a medical diagnostic device. The UI will prominently display the model's classification confidence score, advising users to manually verify the packaging if the confidence threshold drops below 90%.

15.RESOURCE ALLOCATION AND BUDGET ESTIMATION

15.1. Human Resources

As this is an undergraduate academic project for the Faculty of Computing at Sabaragamuwa University of Sri Lanka, the primary human resources consist of the four core team members. Labor costs are effectively zero, as development efforts are contributed in exchange for academic credit.

- Developer A (Mekala): Allocated to Frontend Mobile Development and UI/UX Design.
- Developer B (Rashmika): Allocated to Machine Learning, NLP Pipeline, and Dataset Annotation.
- Developer C (Deemantha): Allocated to Database Architecture and Semantic Translation Logic.
- Developer D (Dilshan): Allocated to Cloud Backend APIs, Edge Integration, and DevOps.

15.2. Hardware Requirements

Development and testing will be conducted utilizing the team members' existing personal hardware, eliminating the need for upfront capital expenditure on physical workstations.

- **Development Workstations:**
  Four standard laptops (minimum 8GB RAM, Intel i5/AMD Ryzen 5 equivalent) capable of running Android Studio emulators, Docker containers, and Jupyter Notebooks locally.

- **Testing Devices:**
  A minimum of two physical smartphone devices (at least one Android and one iOS device) equipped with high-resolution cameras with autofocus capabilities to accurately test the Google ML Kit edge OCR extraction under real-world lighting conditions.

15.3. Software and Cloud Infrastructure

The project architecture strategically relies on open-source frameworks (Flutter, Python, PostgreSQL, scikit-learn) and academic/free-tier cloud services to minimize operational costs during the 15-week development lifecycle.

- **Source Control and CI/CD:**
  GitHub (Free Tier) provides unlimited private repositories and sufficient GitHub Actions minutes for automated testing pipelines.

- **UI/UX Design:**
  Figma (Student/Education Free Tier) utilized for collaborative interface wireframing.

- **Project Management:**
  Jira by Atlassian (Free Tier for small teams under 10 users).

- **Cloud Application Hosting (FastAPI):**
  Deploying a machine learning model into active memory requires slightly more RAM than standard web applications. The team will utilize platforms like Render or Railway. While a free tier may be used for initial API testing, a basic paid tier (approximately $5 - $10 USD per month) is allocated to ensure the container has sufficient memory (1GB - 2GB RAM) to load the spaCy NLP pipeline without crashing during live inference.

- **Cloud Database (PostgreSQL):**
  Managed database providers such as Supabase, Neon, or Render's database tier will be utilized. The free tier limits (typically 500MB storage) are more than sufficient for storing the text-based dietary profiles, allergen mapping schemas, and inference logs.

15.4. Total Estimated Budget

The total financial budget for the duration of the 15-week project is kept strictly under $30 USD. This minimal budget is exclusively reserved for upgrading the cloud backend hosting tier to ensure smooth, low-latency API performance during the final academic evaluation and demonstration phases. All other resources are covered by existing personal hardware and student-tier software licenses.

16.PROJECT DELIVERABLES AND EXPECTED OUTCOMES

16.1. Primary System Deliverables

The successful execution of this project will result in four primary technical deliverables that comprise the new Client-Server architecture:

- **Mobile Edge Client (Flutter):**

  A cross-platform mobile application optimized for Android and iOS, featuring integrated Google ML Kit for on-device OCR extraction, a real-time camera viewfinder, and a dynamic color-coded safety dashboard.

- **Cloud API Backend (FastAPI):**

  A containerized, asynchronous RESTful API that handles JSON Web Token (JWT) authentication, payload validation, and system orchestration.

- **Machine Learning Pipeline (Python):**

  A deployed scikit-learn and spaCy NLP classification model, specifically trained on a proprietary dataset of noisy, real-world Sri Lankan food packaging strings to accurately predict ground-truth ingredient entities.

- **Relational Knowledge Base (PostgreSQL):**

  A fully normalized cloud database hosting multi-tenant user dietary profiles, ML inference feedback logs, and a semantic translation schema for resolving complex allergens and E-numbers.

16.2. Expected Outcomes and Impact

The transition from a theoretical, rule-based offline prototype to a cloud-driven machine learning architecture fundamentally elevates the system's reliability and real-world applicability:

- **Enhanced Detection Accuracy:**

  By replacing static regular expressions with supervised NLP classification, the system will mathematically account for OCR distortion, minimizing life-threatening false negatives.

- **Deep Semantic Intelligence:**

  The complex SQL mapping logic ensures that obscure chemical additives (e.g., E322) and scientific alternative names (e.g., Sodium Caseinate) are automatically and deterministically translated into actionable major allergen warnings (e.g., Soy, Egg, Milk).

- **Adaptive Continuous Learning:**

  The architecture shifts from a static tool to a living system. By automatically recording inference failures and noisy text anomalies into the cloud database, the team establishes a continuous feedback loop to iteratively retrain and improve the ML model over time.

16.3. Future Enhancements

While the current scope focuses on core OCR and NLP classification, the architecture is designed to scale and support several future iterations:

- **Augmented Reality (AR) Overlay:**

  Integrating AR into the Flutter camera viewfinder to physically highlight dangerous ingredients in red directly on the product packaging in real-time.

- **Barcode Integration Fallback:**

  Adding a barcode scanner connected to external open-source databases (like Open Food Facts) as a secondary validation layer when OCR extraction fails due to severely damaged packaging.

- **Multilingual OCR and NLP:**

  Expanding the Google ML Kit configuration and training new spaCy pipelines to support Sinhala and Tamil text extraction, catering specifically to the broader Sri Lankan market.

- **Community Cloud Database:**

  Allowing users to anonymously flag safe and unsafe products, building a crowdsourced, searchable directory of verified allergy-safe products mapped to specific profiles.

CONCLUSION

This software engineering project successfully applies modern architectural paradigms to address a critical healthcare challenge. By deliberately decoupling the mobile interface from the heavy data-processing tier, the system achieves rapid responsiveness at the edge while leveraging the computational power of the cloud for complex natural language processing and relational mapping. The resulting AI-Based Food Label Scanner provides a robust, scalable, and highly accurate decision-support platform, empowering individuals, parents, and caregivers to manage severe dietary restrictions with unprecedented confidence.

REFERENCES

The following technical documentation, academic guidelines, and framework resources were consulted in the architectural design and system planning of this project:

[1] World Health Organization (WHO) and Food and Agriculture Organization (FAO). "Risk Assessment of Food Allergens." WHO Technical Report Series, 2023.

[2] U.S. Food and Drug Administration (FDA). "Food Allergies: What You Need to Know." Center for Food Safety and Applied Nutrition.

[3] Flutter Dev. "Flutter Architectural Overview and On-Device Machine Learning." Official Documentation. https://flutter.dev/docs.

[4] Google Developers. "Google ML Kit: Vision API for Text Recognition V2." Official ML Kit Documentation. https://developers.google.com/ml-kit.

[5] FastAPI. "FastAPI, Python, and AsyncIO: High Performance Web Frameworks." Official Documentation. https://fastapi.tiangolo.com/.

[6] Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, vol. 12, pp. 2825-2830.

[7] Honnibal, M., & Montani, I. "spaCy: Industrial-strength Natural Language Processing in Python." Official Documentation and API Reference. https://spacy.io/.

[8] PostgreSQL Global Development Group. "PostgreSQL Relational Database: High Concurrency and JSONB Handling." Official Documentation. https://www.postgresql.org/docs/.

[9] Atlassian. "Agile Software Development with Scrum and Jira." Atlassian Agile Coach. https://www.atlassian.com/agile.

[10] GitHub. "GitHub Flow and CI/CD with GitHub Actions." GitHub Docs. https://docs.github.com/en/actions.